



ROI Training

Anthropic Models on Amazon Bedrock

Module 7:

AI-Powered Development on Bedrock



Lab 2

Course Contents

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

Module 3: Data Preparation and Knowledge Bases for RAG

Module 4: Tool Use and Agentic Workflows with Claude

Module 5: Security, Governance, and Guardrails

Module 6: Practical Architecture and Deployment Patterns

► **Module 7: Claude Code: Building Bedrock Applications**

Module 8: Amazon Kiro and Enterprise Code Generation



Module Objectives



Install and Configure

Set up Claude Code with Amazon Bedrock as the backend



Agentic Workflows

Scaffold, generate, and debug Bedrock application code



Power Features

Memory, MCP servers, permissions, hooks, extended thinking



Project Intelligence

CLAUDE.md files, settings hierarchy, team conventions

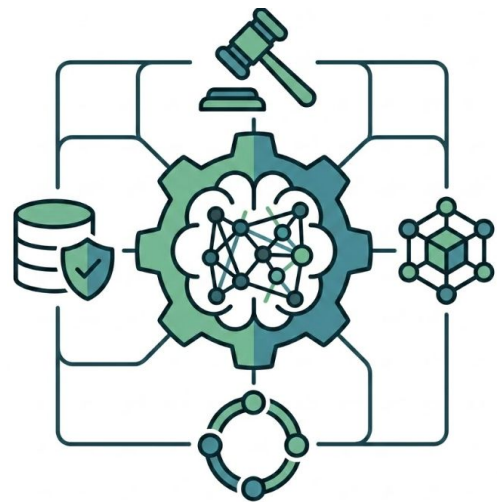


ROI Training

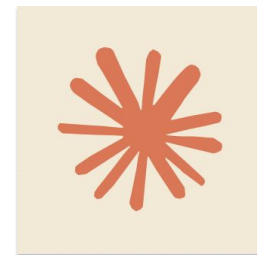
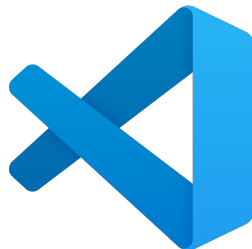
Claude Code

The Agentic Difference

- **Reads** your files, dependencies, and project structure
- **Plans** the approach — which files to create or modify
- **Edits** multiple files in a single operation
- **Runs** shell commands (builds, tests, linters)
- **Iterates** — if a test fails, reads error, fixes code, runs again



Claude Code Interfaces



Terminal / CLI

claude command
Most powerful for AWS
Run shell directly

VS Code

Native extension
Lives in your editor
Most popular interface

JetBrains

IntelliJ, PyCharm,
WebStorm plugin
Full agentic features

Desktop and Web

Mac/Windows app
claude.ai/code
browser
No install required

Four-Stage Development Workflow



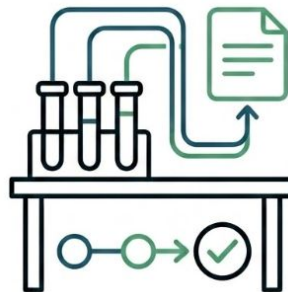
Scaffold

Create project structure, dependencies, config files from a single prompt



Generate

Write Bedrock integration code — boto3 clients, API calls, streaming, error handling



Test

Auto-generate unit tests with mocked AWS services, run and verify



Debug

Paste errors, Claude Code reads code context, fixes the issue, re-runs tests



ROI Training

Amazon Bedrock Setup

Prerequisites

1

AWS account with Bedrock available in your region

2

AWS credentials configured (aws configure or SSO)

3

Node.js 18+ for Claude Code installation

4

IAM principal with bedrock:InvokeModel permission

5

Model access requested for Claude in Bedrock console

Step 1: Install and Enable Claude Code

```
# Install Claude Code globally  
npm install -g @anthropic-ai/claude-code  
claude --version      # confirm installation
```

```
# Verify your AWS credentials work  
aws sts get-caller-identity
```

```
# Then in the AWS Console:  
#   Bedrock → Model access → Enable specific models  
#   Select: Anthropic → Claude Sonnet 4.6, Claude Haiku 4.5  
#   Submit (Anthropic models approve instantly)
```

Step 2: IAM Permissions for Claude Code

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": [
      "bedrock:InvokeModel",
      "bedrock:InvokeModelWithResponseStream"
    ],
    "Resource": [
      "arn:aws:bedrock:*::foundation-model/anthropic.claude-*"
    ]
  }]
}
```

Step 3: Configure Environment Variables

```
# Add to ~/.bashrc, ~/.zshrc, or ~/.profile
```

```
export CLAUDE_CODE_USE_BEDROCK=1
```

```
export AWS_REGION=us-east-1
```

```
# Pin model versions (recommended for stability)
```

```
export ANTHROPIC_DEFAULT_SONNET_MODEL='us.anthropic.claude-sonnet-4-6'
```

```
export
```

```
ANTHROPIC_DEFAULT_HAIKU_MODEL='us.anthropic.claude-haiku-4-5-20251001-v1:0'
```

```
# Reload your shell
```

```
source ~/.zshrc
```

Step 4: Verify and First Run

```
# Confirm environment is set
echo $CLAUDE_CODE_USE_BEDROCK      # should print: 1
echo $AWS_REGION                    # should print: us-east-1

# Start Claude Code in your project directory
cd ~/projects/my-bedrock-app
claude

# In Claude Code, send a test prompt:
#   > what does this project do?
#
# If you get a response, Bedrock is wired up correctly.
# Check CloudTrail to confirm the invocation logged.
```

Why Run Claude Code on Bedrock?

- **Security:** All AI traffic stays within your AWS account
- **Compliance:** CloudTrail logs every interaction
- **Cost control:** Token usage in AWS Cost Explorer
- **Governance:** IAM policies control model access
- **No extra licensing:** Uses existing Bedrock model access





ROI Training

Dev Workflows

Workflow: Scaffold an Amazon Bedrock App

You: Create a Lambda function that queries our Knowledge Base and returns cited results. Use the Converse API.

Claude Code:

1. Reads your project structure and existing patterns
2. Creates `lambda_handler.py` with Bedrock client
3. Implements `retrieve_and_generate` with your KB ID
4. Adds citation extraction and formatting
5. Creates SAM template for deployment
6. Writes test cases with mock responses
7. Runs tests – all pass

Workflow: Add Guardrails to Existing Code

You: Add Bedrock Guardrails to our chat handler.
Use guardrail ID 'abc123' version '1'.
Apply to both input and output.

Claude Code:

1. Reads your existing chat handler
2. Adds ApplyGuardrail calls before and after invoke
3. Implements blocked response handling
4. Updates .env template with GUARDRAIL_ID
5. Adds guardrail test cases
6. Runs tests to verify nothing broke

Workflow: Debug Amazon Bedrock Issues

You: I'm getting `ThrottlingException` on streaming calls.
Here's the error: `[Claude Code reads your file]`

Claude Code:

1. Analyzes your retry configuration
2. Identifies: no backoff on streaming calls
3. Implements exponential backoff with jitter
4. Adds circuit breaker for sustained throttling
5. Creates test simulating throttling
6. Runs tests to verify the fix

Workflow: Rapid Prototyping

You: Prototype a content moderation pipeline:

- Accept text via API Gateway
- Run through Bedrock Guardrail
- If blocked, log violation, return safe response
- If passed, process with Claude Sonnet
- Emit CloudWatch metrics for both outcomes

Claude Code creates in ~2 minutes:

- ✓ moderation_pipeline.py
- ✓ config.py
- ✓ monitoring.py
- ✓ test_moderation.py
- ✓ requirements.txt
- ✓ template.yaml (SAM)

Workflow: Node.js Amazon Bedrock Client

You: Create a Node.js module for invoking Claude via Bedrock using AWS SDK v3. Include streaming and retry logic.

Claude Code:

1. Creates `src/bedrock-client.js` with `@aws-sdk/client-bedrock-runtime`
2. Implements `InvokeModelWithResponseStream`
3. Adds async iterator for streaming chunks
4. Retry with exponential backoff
5. TypeScript types for request/response
6. Jest tests with AWS SDK mocks

Effective Prompting for Claude Code

- Be specific about AWS services: "Use boto3 bedrock-runtime client"
- Specify error handling: "Include retry for ThrottlingException"
- Reference existing code: "Follow the pattern in bedrock.py"
- Name the API: "Use the Converse API, not Messages API"
- Include constraints: "Max 1000 tokens, us-east-1 region"
- Ask for tests: "Write tests with mocked Bedrock responses"

Iterative Development Pattern

1

Generate initial code — basic Bedrock client invocation

2

Add features — streaming and connection pooling

3

Harden — retry on `ThrottlingException`, timeouts

4

Test — success, timeout, and throttling scenarios

5

Deploy — SAM template for Lambda

Additional Capabilities

Tool Use and RAG

Generate tool definitions,
query Knowledge Bases,
inject retrieved context

Multimodal

Process images alongside
text for document
analysis pipelines

Infra as Code

Generate CloudFormation,
SAM, and CDK templates
from requirements

CI/CD and Git

Headless mode for code
review, automated commits,
branches, and PRs



ROI Training

Power Features

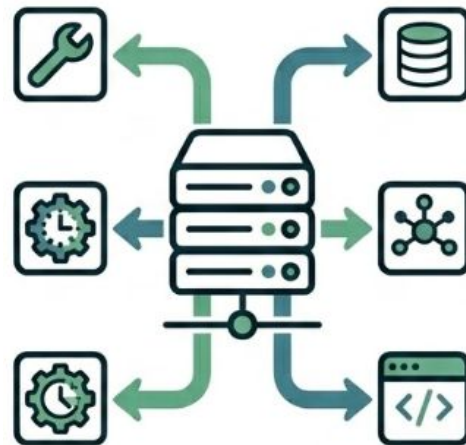
Memory — Persistent Context

- Claude Code remembers context across sessions
- Auto-saves: project patterns, decisions, preferences
- KB IDs, guardrail configs persist — no repeating yourself
- Location:
~/.claude/projects/<your-project>/memory/
- Close terminal, come back tomorrow — it remembers



MCP Servers — External Connections

- MCP = Model Context Protocol
- Connect Claude Code to databases, APIs, AWS services
- Read CloudWatch metrics directly
- Query DynamoDB tables
- Fetch CloudFormation stack outputs



Permissions and Modes

Default Mode

Ask permission
before every
action

Auto Mode

Approve safe actions,
ask for
risky ones

Plan Mode

Create plan first,
you approve,
then execute

Team Settings

.claude/settings.json
committed to repo —
shared safety rules

Hooks — Automation Triggers

```
// .claude/settings.json
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write|Edit",
      "hooks": [{
        "type": "command",
        "command": "python -m pytest tests/ -q"
      }]
    }]
  }
}
```

Extended Thinking

- Toggle with /thinking command
- Claude reasons through problems before responding
- Best for: architecture design, complex debugging, security reviews
- Produces significantly better results for complex tasks
- Up to 1M token context — reads your entire codebase



ROI Training

Project Intelligence

CLAUDE.md — Team Conventions

Bedrock Customer Support Application

Conventions

- Use Bedrock Converse API (not Messages API)
- All functions must have type hints and docstrings
- Use structured logging with `aws_lambda_powertools`

Security — NON-NEGOTIABLE

- Never hardcode credentials or API keys
- All IAM policies must follow least-privilege
- Guardrail ID must not appear in client-side code

Testing

- Every new function needs unit tests
- Mock Bedrock responses using `botocore.stub`

Settings Hierarchy

1

~/.claude/settings.json — your personal prefs (not committed)

2

.claude/settings.json — team hooks and permissions (committed)

3

.claude/settings.local.json — personal overrides (gitignored)

4

Project settings go in git — same config for the whole team

5

Personal preferences stay local to your machine

Best Practices

- **Pin model versions** — `us.anthropic.claude-sonnet-4-6`
- **Review security code** — always review IAM policies, guardrails
- **Start with explanation** — understand before changing
- **Write tests alongside** — let Claude Code write tests with features

Best Practices

- **Use CLAUDE.md** — team conventions, architecture decisions
- **Set up hooks** — auto-test, auto-lint on every edit
- **Configure permissions** — lock down destructive commands

Common Pitfalls to Avoid

- Accepting code without review
 - Always validate IAM policies and security
- Vague prompts yield generic code
 - Be specific about patterns and services
- Not leveraging context
 - Reference existing files for consistency

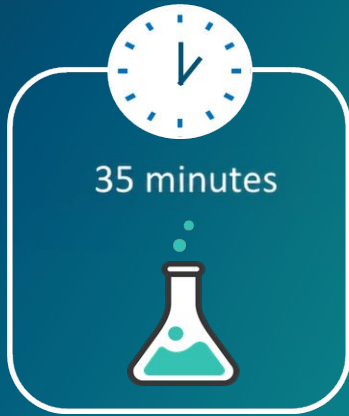
Common Pitfalls to Avoid

- Skipping tests
 - Always ask for tests alongside implementation
- Not using CLAUDE.md
 - Team conventions get lost without it
- Ignoring model pinning
 - Unpinned versions break when models update

Module 8 Summary

- Claude Code is an **agent** that takes actions, not just generates text
- Connect to **Amazon Bedrock** for enterprise governance and compliance
- **Agentic workflows**: scaffold, integrate, debug, prototype
- **Memory** persists context, no repeating your setup
- **MCP** connects to AWS services
- **CLAUDE.md + Hooks + Permissions** = team productivity

Lab 2



Claude Code on Bedrock

In this 35m mini lab, you will:

- Install Claude Code for Amazon Bedrock
- Use [CLAUDE.md](#) to enforce coding and writing standards
- Scaffold an Amazon Bedrock function using natural language
- Experience agentic workflows, memory systems and collaborative techniques
- Review production deployment strategies with development containers

What We Covered



Install and Configure

Set up Claude Code with Amazon Bedrock as the backend



Agentic Workflows

Scaffold, generate, and debug Bedrock application code



Power Features

Memory, MCP servers, permissions, hooks, extended thinking



Project Intelligence

CLAUDE.md files, settings hierarchy, team conventions

Module 8 Is Next!

Module 1: Foundations: Generative AI and Claude Models on AWS

Module 2: Claude API Fundamentals and Prompt Engineering

Module 3: Data Preparation and Knowledge Bases for RAG

Module 4: Tool Use and Agentic Workflows with Claude

Module 5: Security, Governance, and Guardrails

Module 6: Practical Architecture and Deployment Patterns

Module 7: Claude Code: Building Bedrock Applications

► **Module 8: Amazon Kiro and Enterprise Code Generation**

